ASADITO RG

GUÍA DE IMPLEMENTACIÓN MANUAL

Ampliaciones del backend

Movimientos, presupuestos guardados y verificaciones.
Confirmación de emails y chatbot de WhatsApp opcional.

Tu punto de partida
Java 11 · Spring Boot 2.7.14 · PostgreSQL
Sesiones actuales y DTO originales.

Una etapa por vez: tablas, relaciones, código clave, permisos y pruebas.

ASADITO RG

11 de noviembre de 2026 · Base de código: 4d58c55

Propuesta para codificar a mano. La aplicación no fue modificada.

Contenido

Usá el índice para avanzar por etapas. Los enlaces internos también funcionan en el PDF.

Propuesta adaptada al commit 4d58c55. Los fragmentos se completan a mano; las funciones descriptas todavía no se agregaron a la aplicación.

Mapa de relaciones 3

1. Orden de trabajo y mapa del proyecto 4

2. Tabla movimientos: quién hizo cada cambio 6

3. Tabla presupuestos: guardar una lista del usuario 9

4. Tabla verificaciones: comparar estimado y pagado 14

5. Confirmación de emails con la sesión existente 17

6. WhatsApp opcional: empezar con un bot de menú 23

7. Endpoints y permisos para revisar antes de programar 26

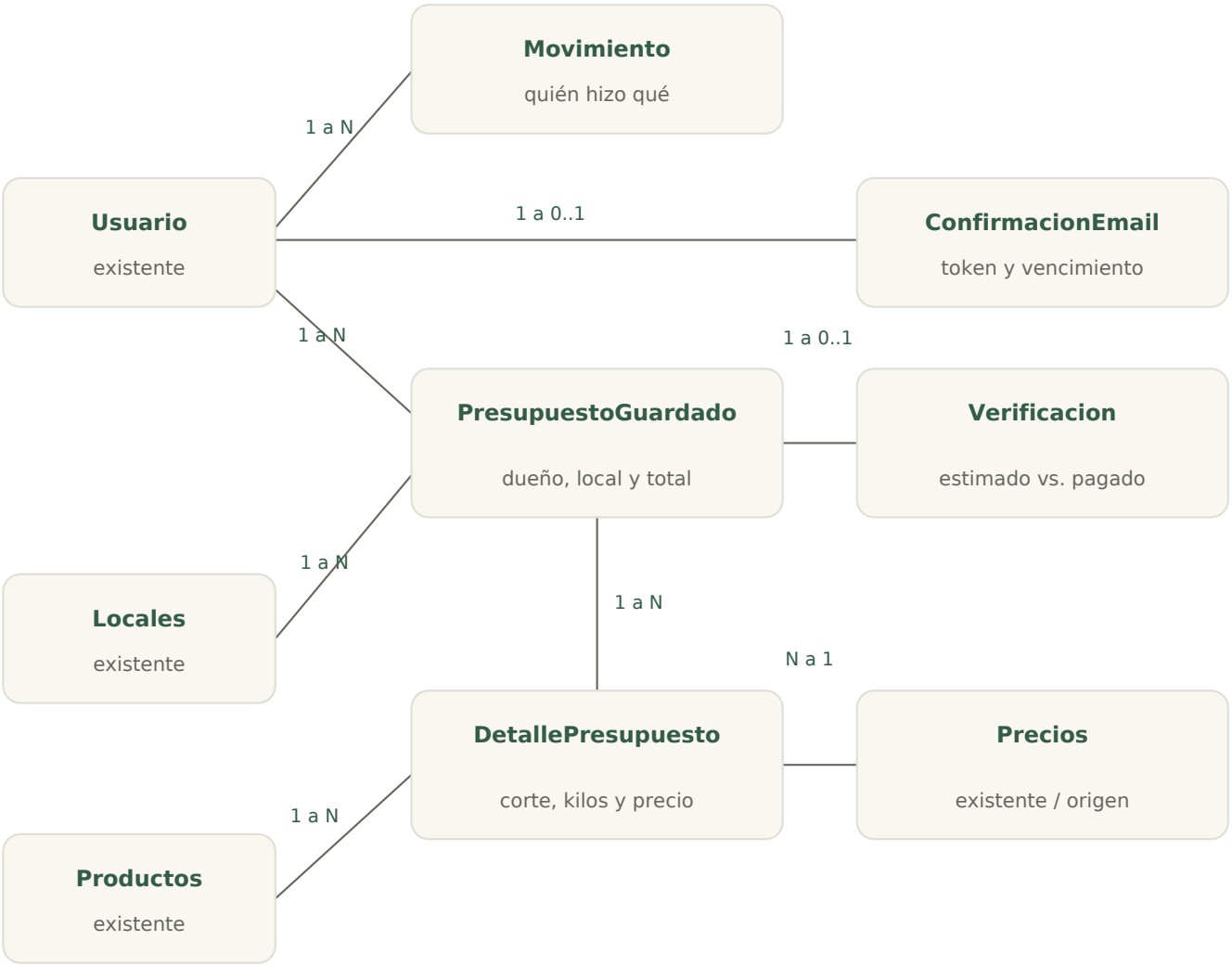
8. Crear tablas y avanzar sin perder datos 27

9. Pruebas concretas y criterio de terminado 29

10. Cómo explicarlo y lecturas para continuar 31

Mapa de relaciones

Las líneas muestran las relaciones nuevas. “1 a N” representa uno a muchos; “1 a 0..1”, una relación opcional de uno a uno.



Precios ya relaciona Locales y Productos en tu código original; esas relaciones no se vuelven a dibujar. El detalle conserva su precio de origen y una copia de los valores usados.

El vínculo al usuario identifica al dueño. El presupuesto guardado tiene detalles; su verificación compara el total histórico con un pago declarado. La confirmación de email pertenece a la cuenta, no al presupuesto.

1. Orden de trabajo y mapa del proyecto

Implementá un módulo por vez. Primero comprobá que arranca y que sus permisos funcionan; después continuá con el siguiente. Los fragmentos muestran las partes importantes, no archivos completos listos para pegar: completá imports, constructores y accesores donde se indica.

Orden	Módulo	Resultado para probar
1	Movimientos	Al publicar un precio queda registrado quién lo hizo.
2	Presupuestos y sus detalles	Una persona guarda su lista y vuelve a verla sin que cambien sus importes.
3	Verificaciones	La persona informa cuánto pagó y el sistema clasifica la diferencia.
4	Confirmación de email	Una cuenta nueva se habilita mediante un enlace de un solo uso.
5	WhatsApp, opcional	Un bot ofrece un menú y consultas; las funciones privadas requieren vinculación de cuenta.

Todas las rutas Java parten de src/main/java/com/appasadito/AsaditoRg/. Usá los paquetes modelos, repositorios, controllers y services que ya existen. Los archivos nuevos usan nombres en singular para distinguirlos de los DTO originales.

Los DTO LocalesDTO, ProductosDTO, PreciosDTO y ProductoPrecioDTO permanecen como están. Estas ampliaciones no requieren cambiar sus campos ni reemplazar el servicio original de precios. Para las respuestas nuevas se muestran Map y listas de Map pequeños; más adelante podés crear DTO exclusivos de estas funciones si te resultan más claros.

Tablas y relaciones propuestas

Tabla nueva	Relación principal	Para qué sirve
movimientos	Muchos movimientos corresponden a un usuario.	Auditoría de acciones realizadas desde el servidor.
presupuestos	Muchos presupuestos pertenecen a un usuario y un local.	Cabecera, fecha, comercio y total histórico.
presupuesto_detalles	Muchos detalles pertenecen a un presupuesto.	Corte, kilos, precio aplicado y subtotal.
verificaciones	Un presupuesto tiene como máximo una verificación.	Total pagado y resultado de la comparación.
confirmaciones_email	Un usuario tiene como máximo un registro de confirmación vigente o consumido.	Hash del token, vencimiento y uso.

Se agrega email_confirmado a usuarios. Las tablas usuarios, productos, locales y precios ya existen. El detalle del presupuesto es una tabla adicional necesaria: una fila de presupuesto por sí sola no puede representar varios cortes con cantidades distintas de manera ordenada.

Convenciones para mantenerlo sencillo

• IDs nuevos: Long con GenerationType.IDENTITY. Los IDs existentes de productos, locales y precios siguen siendo int; el de Usuario sigue siendo Long.

• Fechas de eventos: Instant. Para la fecha del precio se conserva LocalDate. En application.properties podés agregar spring.jpa.properties.hibernate.jdbc.time_zone=UTC; en la interfaz mostrás la hora de Argentina.

• Importes nuevos: BigDecimal, columna numeric(14,2). Cantidades: BigDecimal, numeric(5,1), con máximo 1000 kg y pasos de 0,5 kg.

• Nuevas entidades con constructor vacío y getters/setters. Podés generarlos con IntelliJ: Alt+Insert o Code → Generate. No hace falta Lombok.

• No agregues colecciones de estas entidades a Usuario. Es suficiente que cada nueva fila tenga su referencia al usuario; eso reduce relaciones y evita ciclos JSON.

• Las escrituras relacionadas se realizan en una transacción. El servidor toma el usuario de Authentication, nunca de un usuarioId enviado por el formulario.

Importaciones frecuentes: javax.persistence.*, java.math.BigDecimal, java.math.RoundingMode, java.time.Instant, java.time.LocalDate, org.springframework.transaction.annotation.Transactional y org.springframework.web.server.ResponseStatusException. Usá javax, no jakarta, con la versión actual del proyecto.

2. Tabla movimientos: quién hizo cada cambio

Un movimiento es un registro de auditoría. Ejemplo: “el usuario 7 publicó el precio 120 para el corte 3 y local 5”. La pantalla de consulta es opcional; guardar correctamente el movimiento es lo primero.

Modelo Movimiento.java

```
@Entity
@Table(name = "movimientos")
public class Movimiento {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "usuario_id")
    private Usuario usuario;

    @Column(nullable = false)
    private Instant fecha = Instant.now();
    @Column(nullable = false, length = 40)
    private String accion;
    @Column(nullable = false, length = 40)
    private String entidad;
    @Column(nullable = false)
    private Long registroId;
    @Column(nullable = false, length = 300)
    private String resumen;

    public Movimiento() {}
    // Generá getters y setters con IntelliJ.
}
```

El usuario puede ser null únicamente para acciones del sistema o acciones públicas sin sesión. En las operaciones de gestión, presupuesto y verificación debe ser el usuario autenticado. No aceptes ese campo desde el frontend. Los eventos públicos, si los registrás, se muestran como SISTEMA; no inventes una cuenta administradora para representarlos.

Acciones sugeridas: CORTE_CREADO, CORTE_MODIFICADO, CORTE_BAJA, LOCAL_CREADO, LOCAL_MODIFICADO, LOCAL_BAJA, PRECIO_PUBLICADO, OPERADOR_CREADO, PRESUPUESTO_CREADO, VERIFICACION_CREADA y EMAIL_CONFIRMADO. Elegilas en Java; no son texto libre del cliente.

Repositorio y servicio

Creá MovimientoRepository extends JpaRepository<Movimiento, Long>. Agregá el método para consultar sin cargar toda la tabla:

```
Page<Movimiento> findAllByOrderByFechaDescIdDesc(Pageable pageable);
```

Page, Pageable y PageRequest pertenecen a org.springframework.data.domain. El servicio recibe al actor que ya resolvió el controlador:

```
@Service
public class MovimientoService {
    @Autowired private MovimientoRepository movimientos;
```

```
    public void registrar(Usuario actor, String accion,
                          String entidad, Long registroId,
                          String resumen) {
        Movimiento m = new Movimiento();
        m.setUsuario(actor);
        m.setAccion(accion);
        m.setEntidad(entidad);
        m.setRegistroId(registroId);
        m.setResumen(resumen);
        movimientos.save(m);
    }
}
```

Integrarlo a PreciosController

Agregá UsuarioRepository y MovimientoService con @Autowired. En el método agregar que ya existe incorporá Authentication authentication como último parámetro y @Transactional sobre el método público. Conservá las validaciones actuales. Reemplazá solamente el guardado/retorno final por:

```
Precios guardado = preciosrepository.save(nuevo);
Usuario actor = usuarioRepository.findByEmail(authentication.getName());
movimientoService.registrar(actor, "PRECIO_PUBLICADO", "Precios",
    (long) guardado.getId(),
    "Corte " + productoId + ", local " + localId);
return new PreciosDTO(guardado);
```

La transacción debe abarcar tanto save del precio como registrar el movimiento. Sin @Transactional en el método exterior, el repositorio puede confirmar primero el precio y después fallar la auditoría. Con la transacción exterior, si falla cualquiera de los dos guardados se revierte todo. No captures y ocultes la excepción del movimiento.

Repetí la integración en los métodos públicos agregar/modificar/eliminar de ProductosController y LocalesController. En la baja se registra la acción después de setActivo(false) y save, dentro de la misma transacción. Para “modificó X” basta una descripción corta; registrar valores anteriores/nuevos puede quedar como mejora posterior.

Nunca guardes contraseñas, hashes de contraseñas, tokens de confirmación ni cuerpos completos de solicitudes en resumen. Tampoco registres un email enviado como si ya estuviera confirmado.

Consultarlo desde ADMIN

Proponé GET /api/admin/movimientos?page=0&size=20. En WebAuthorization, ANTES de .antMatchers("/api/admin/**").denyAll(), agregá:

```
.antMatchers(HttpMethod.GET, "/api/admin/movimientos")
    .hasAuthority("ADMIN")
```

Validá page >= 0 y size entre 1 y 100; usá PageRequest.of(page, size). Desde un método @Transactional(readOnly = true), convertí cada fila a un Map con id, fecha, usuarioId, accion, entidad, registroId y resumen. Si no hay actor, para el campo visible usá el texto SISTEMA. Map.of no acepta null: omití usuarioId o usá un HashMap si querés devolver null.

No devuelvas Movimiento directamente: su relación LAZY puede estar cerrada al serializar y no hace falta exponer toda la entidad Usuario. La API no debe tener POST/PUT/DELETE públicos de movimientos; se generan únicamente como consecuencia de acciones reales del servidor.

Probalo: publicar como OPERADOR crea una fila; un precio inválido no crea ninguna; USER y OPERADOR reciben 403 al consultar auditoría; ADMIN puede verla. Una cuenta anónima no puede crear auditoría ni publicar precios.

3. Tabla presupuestos: guardar una lista del usuario

La propuesta inicial es un presupuesto por comercio. La comparación de tres comercios sigue siendo pública en el frontend; el usuario elige uno y guarda esa opción. Guardar no reserva mercadería ni congela el precio comercial: conserva una estimación histórica.

La cabecera contiene el dueño y el comercio. Cada detalle conserva una copia del nombre del corte, kilos, precio por kilo, fecha del precio y subtotal. Estas copias se llaman snapshots: si mañana cambia el catálogo, el presupuesto guardado sigue mostrando lo que la persona vio al guardarlo.

Modelo PresupuestoGuardado.java

```
@Entity
@Table(name = "presupuestos")
public class PresupuestoGuardado {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "usuario_id", nullable = false)
    private Usuario usuario;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "local_id", nullable = false)
    private Locales local;

    @Column(nullable = false)
    private Instant fecha = Instant.now();
    @Column(nullable = false, length = 120)
    private String localNombre;
    @Column(nullable = false, length = 200)
    private String localDireccion;
    @Column(nullable = false, length = 3)
    private String moneda = "ARS";
    @Column(nullable = false, precision = 14, scale = 2)
    private BigDecimal total;

    @OneToMany(mappedBy = "presupuesto", cascade = CascadeType.PERSIST)
    @OrderBy("id ASC")
    private List<DetallePresupuesto> detalles = new ArrayList<>();

    public PresupuestoGuardado() {}
    public void agregarDetalle(DetallePresupuesto detalle) {
        detalle.setPresupuesto(this);
        detalles.add(detalle);
    }
    // Generá getters/setters; no hace falta reemplazar detalles.
}
```

List y ArrayList vienen de java.util. No uses CascadeType.REMOVE hacia Usuario, Locales o Productos: borrar un presupuesto no debe borrar al comercio ni a su dueño. En esta primera versión no se ofrece eliminación ni modificación de presupuestos guardados.

Modelo DetallePresupuesto.java

```
@Entity
@Table(name = "presupuesto_detalles")
public class DetallePresupuesto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "presupuesto_id", nullable = false)
    private PresupuestoGuardado presupuesto;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "producto_id", nullable = false)
    private Productos producto;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "precio_id", nullable = false)
    private Precios precioOrigen;

    @Column(nullable = false, length = 120)
    private String corteNombre;
    @Column(nullable = false, precision = 5, scale = 1)
    private BigDecimal cantidad;
    @Column(nullable = false, precision = 14, scale = 2)
    private BigDecimal precioKg;
    @Column(nullable = false)
    private LocalDate fechaPrecio;
    @Column(nullable = false, precision = 14, scale = 2)
    private BigDecimal subtotal;

    public DetallePresupuesto() {}
    // Generá getters y setters.
}
```

No necesitás un repositorio de detalles para esta etapa: se guardan mediante agregarDetalle y el cascade PERSIST de la cabecera. Las referencias permiten rastrear el origen y las copias conservan el contenido histórico. Las bajas de cortes/locales siguen siendo lógicas; no se borran filas que otros registros referencian.

Repositorios

Creá PresupuestoRepository extends JpaRepository<PresupuestoGuardado, Long> con:

```
Page<PresupuestoGuardado> findByUsuario_IdOrderByFechaDescIdDesc(
    Long usuarioId, Pageable pageable);
Optional<PresupuestoGuardado> findByIdAndUsuario_Id(
    Long id, Long usuarioId);
```

Optional viene de java.util. En el PreciosRepository original agregá un método, sin quitar su consulta actual:

```
Optional<Precios>
findFirstByLocal_IdAndProducto_IdAndActivoTrueAndFechaLessThanEqualOrderByFechaDescIdDesc(
    int localId, int productoId, LocalDate fecha);
```

Es un nombre largo, pero Spring lo interpreta: mismo local y corte, activo, fecha no futura, orden descendente por fecha e ID y solo la primera coincidencia. El controlador debe comprobar además que el corte y el local estén activos. No uses obtenerUltimosPrecios para guardar: la respuesta original agrupa por nombres y no resuelve por ID todos los empates.

POST /api/presupuestos: contrato del formulario

Usá un nuevo PresupuestoController. El formulario envía SOLO localId y listas repetidas de productoId y cantidad:

```
localId=3&productoId=1&cantidad=1.5&productoId=2&cantidad=0.5
```

La firma del método puede ser:

```
@PostMapping("/api/presupuestos")
@ResponseStatus(HttpStatus.CREATED)
@Transactional
public Map<String, Object> guardar(
        @RequestParam int localId,
        @RequestParam List<Integer> productoId,
        @RequestParam List<BigDecimal> cantidad,
        Authentication authentication) {
    // Implementá los pasos que se detallan a continuación.
}
```

Si ponés @RequestMapping("/api/presupuestos") en la clase, usá @PostMapping sin repetir la ruta. Este fragmento de método es un esqueleto de trabajo: completá su cuerpo y retorno antes de compilar.

1. Resolvé Usuario por authentication.getName(). No recibas usuarioId, email, total, precioKg ni nombre de comercio como datos confiables del formulario.

2. Validá que haya de 1 a 100 filas, que ambas listas tengan igual tamaño y que no se repita productoId. Podés comparar el tamaño de new HashSet<>(productoId) con el de la lista.

3. Buscá el local por ID y verificá isActivo; ante ausencia o baja, devolvé 404.

4. Creá PresupuestoGuardado y asigná usuario, local, localNombre y localDireccion tomados de la base.

5. Por cada posición i, validá kilos y buscá el producto activo. Obtené el último precio con el método del repositorio y LocalDate.now(). Si falta, devolvé 400 y revertí todo.

6. Calculá subtotal en Java, creá DetallePresupuesto y completá todas sus propiedades. Usá agregarDetalle para vincular ambos lados de la relación.

7. Sumá los subtotales, validá el límite de numeric(14,2), asigná total y guardá la cabecera. El cascade guarda los detalles.

8. Registrá PRESUPUESTO_CREADO con el usuario autenticado. Devolvé id, total y moneda mediante Map.of. Todo ocurre en la misma transacción.

Cálculo de kilos e importes

```
BigDecimal kilos = cantidad.get(i);
if (kilos == null || kilos.signum() <= 0
        || kilos.compareTo(new BigDecimal("1000")) > 0
        || kilos.remainder(new BigDecimal("0.5")).signum() != 0) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Usá cantidades de 0,5 kg, hasta 1000 kg");
}
kilos = kilos.setScale(1, RoundingMode.UNNECESSARY);

BigDecimal precioKg = new BigDecimal(
    Float.toString(precio.getPrecio()))
    .setScale(2, RoundingMode.HALF_UP);
BigDecimal subtotal = precioKg.multiply(kilos)
    .setScale(2, RoundingMode.HALF_UP);
```

precio es el Precios recuperado de la base. Rechazá también un precio no finito o <= 0 antes de convertirlo. Inicializá total con new BigDecimal("0.00"), y acumulá total = total.add(subtotal). El máximo almacenado para total/subtotal es 999999999999.99. Rechazá un total mayor antes del guardado.

BigDecimal es inmutable: add no cambia el objeto original si no asignás el resultado. Construí decimales con texto, no con new BigDecimal(0.1). La conversión desde Float.toString evita añadir decimales binarios innecesarios, pero NO recupera centavos que el float original ya perdió. Se mantiene ese límite del catálogo sin modificar tus DTO ni el modelo de precios.

El frontend puede seguir calculando una vista previa, pero el servidor recalcula al guardar. Si el precio cambió mientras la persona miraba la página, mostrale el total guardado y actualizá el reporte con ese resultado. No prometas que coincidirá con una vista previa antigua.

Consultar únicamente los presupuestos propios

GET /api/presupuestos lista las cabeceras del usuario actual, paginadas. GET /api/presupuestos/{id} busca por id Y usuarioId. Este control es obligatorio aunque conozcas el ID y aunque la ruta exija sesión:

```
Usuario usuario = usuarios.findByEmail(authentication.getName());
PresupuestoGuardado p = presupuestos
    .findByIdAndUsuario_Id(id, usuario.getId())
    .orElseThrow(() -> new ResponseStatusException(
        HttpStatus.NOT_FOUND, "Presupuesto no encontrado"));
```

Usá @Transactional(readOnly = true) en las lecturas que recorran detalles LAZY. Armá Maps con los campos públicos: cabecera, nombre/dirección históricos, total y detalles con corteNombre, cantidad, precioKg, fechaPrecio y subtotal. No serialices las entidades completas ni la relación usuario. Un ADMIN tampoco obtiene automáticamente acceso a los presupuestos privados ajenos.

Las rutas quedan autenticadas por la regla .anyRequest().authenticated() actual. Podés añadir una regla explícita para /api/presupuestos y /api/presupuestos/ **antes del final para que sea más legible. No las ubiques en /api/auth/**, que hoy está permitido al público.

Conexión mínima desde el frontend

Para listas repetidas usá URLSearchParams directamente: JSON.stringify de un array no equivale a repetir campos de formulario. Con el helper api que ya tiene la aplicación:

```
const form = new URLSearchParams();
form.set("localId", localId);
items.forEach(item => {
```

```
  form.append("productoId", item.productoId);
  form.append("cantidad", item.cantidad);
});
const guardado = await api("/api/presupuestos", {
  method: "POST", body: form
});
```

Agregá luego “Mis presupuestos” y un botón Guardar para la opción elegida. Para un PDF histórico, obtené GET /api/presupuestos/{id} y adaptá esa respuesta a reporte.js; no vuelvas a calcular con los precios actuales del catálogo. El PDF puede seguir generándose en JavaScript.

Probalo: guardá 1,5 kg, anotá el total y publicá otro precio. Al abrir el guardado debe seguir el importe original. Si un segundo usuario pide el mismo ID, debe recibir 404. Un total enviado manualmente por el cliente no altera el valor calculado por Java.

4. Tabla verificaciones: comparar estimado y pagado

Propuesta de negocio: el dueño del presupuesto informa cuánto pagó por los MISMOS cortes, cantidades y moneda. El sistema compara ese total con el presupuesto guardado. Si cambió la lista, mezcló comercios o agregó envío, el resultado no mide correctamente el cálculo original: pedile que lo aclare o que use una compra comparable.

Esta verificación es una declaración del usuario, no una confirmación oficial del comercio. Como primera versión se permite una sola por presupuesto, sin editar ni eliminar. Esto reduce tablas, endpoints y conflictos.

Modelo Verificacion.java

```
@Entity
@Table(name = "verificaciones")
public class Verificacion {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "presupuesto_id", nullable = false,
                unique = true)
    private PresupuestoGuardado presupuesto;

    @Column(nullable = false)
    private Instant fecha = Instant.now();
    @Column(nullable = false, precision = 14, scale = 2)
    private BigDecimal totalReal;
    @Column(nullable = false, precision = 14, scale = 2)
    private BigDecimal diferencia;
    @Column(nullable = false, precision = 18, scale = 2)
    private BigDecimal porcentaje;
    @Column(nullable = false, length = 20)
    private String resultado;
    @Column(length = 300)
    private String observacion;

    public Verificacion() {}
    // Generá getters y setters.
}
```

No hace falta otro usuario_id: el dueño se conoce a través de presupuesto.usuario. Tampoco hace falta copiar otra vez el estimado: la cabecera guardada no se modifica. El porcentaje tiene más dígitos que el importe porque una diferencia enorme respecto de un estimado pequeño puede producir un porcentaje grande.

En VerificacionRepository extends JpaRepository<Verificacion, Long> agregá:

```
boolean existsByPresupuesto_Id(Long presupuestoId);
Optional<Verificacion> findByPresupuesto_Id(Long presupuestoId);
```

La restricción UNIQUE de presupuesto_id es necesaria además del exists. Dos solicitudes simultáneas pueden pasar el exists antes de guardar: la base evita que queden dos verificaciones. El conflicto devuelve 409 y la segunda operación no genera movimiento.

Fórmula y nombres sin ambigüedad

```
Diferencia = total pagado - total estimado
Porcentaje = diferencia / total estimado × 100
```

Se propone una tolerancia inicial del 5 %, configurable como una constante. ACORDE significa diferencia absoluta de hasta el 5 %, inclusive. SOBREESTIMADO significa que el presupuesto calculó de más; SUBESTIMADO significa que calculó de menos.

Estimado	Pagado	Diferencia	Porcentaje	Resultado
$20.000	$20.800	+$800	+4 %	ACORDE
$20.000	$21.000	+$1.000	+5 %	ACORDE, límite incluido
$20.000	$18.000	-$2.000	-10 %	SOBREESTIMADO: se presupuestó de más
$20.000	$22.000	+$2.000	+10 %	SUBESTIMADO: se presupuestó de menos

El signo está definido respecto del pago real. No lo inviertas al nombrar el resultado. Podés mostrar también “Pagaste menos” o “Pagaste más” en la interfaz.

Cálculo en Java

```
BigDecimal diferencia = totalReal.subtract(estimado);
BigDecimal porcentaje = diferencia.multiply(new BigDecimal("100"))
    .divide(estimado, 2, RoundingMode.HALF_UP);

boolean acorde = diferencia.abs().multiply(new BigDecimal("100"))
    .compareTo(estimado.multiply(new BigDecimal("5"))) <= 0;

String resultado;
if (acorde) {
    resultado = "ACORDE";
} else if (diferencia.signum() < 0) {
    resultado = "SOBREESTIMADO";
} else {
    resultado = "SUBESTIMADO";
}
```

estimado es p.getTotal(), leído de la base y siempre mayor que cero. No calcules si fuera cero: rechazá ese registro como inconsistente. El porcentaje se redondea para guardarlo/mostrarlo, pero la clasificación compara los valores sin redondear el porcentaje. Así, 5,004 % no se vuelve incorrectamente ACORDE al mostrarse como 5,00 %.

Endpoint propuesto

POST /api/presupuestos/{id}/verificacion recibe totalReal y observacion como formulario; GET en la misma ruta permite consultar el resultado. Usá un VerificacionController separado, Authentication y @Transactional para el POST.

1. Buscá el presupuesto con findByIdAndUsuario_Id. Una persona ajena recibe 404, aunque sea ADMIN.

2. Rechazá otra verificación existente con 409.

3. Validá totalReal > 0, no mayor a 999999999999.99 y con hasta dos decimales. Para no redondear un dato ingresado silenciosamente, usá setScale(2, RoundingMode.UNNECESSARY) y devolvé 400 si lanza ArithmeticException.

4. Limitá observacion a 300 caracteres. Indicá que no incluya datos personales innecesarios.

5. Calculá diferencia, porcentaje y resultado en Java. No recibas resultado, porcentaje ni estimado como valores confiables del frontend.

6. Guardá Verificacion, registrá VERIFICACION_CREADA y devolvé sus campos mediante Map. Usá texto vacío si observacion es null para poder usar Map.of.

Probalo con los cuatro casos de la tabla, con una diferencia apenas superior al 5 %, con un presupuesto ajeno y con dos envíos simultáneos. Si la escritura falla, no debe quedar una auditoría de una verificación que no existe.

5. Confirmación de emails con la sesión existente

El flujo propuesto usa un enlace con un token aleatorio: se guarda solamente su hash, vence a los 30 minutos y se consume una vez. No se envía la contraseña por email y no se necesita reemplazar Spring Security ni usar JWT.

Paso A: agregar estado al usuario

En Usuario.java agregá un boolean emailConfirmado con getter y setter. Para la adopción sin bloquear cuentas existentes, esta guía propone considerarlas habilitadas por migración. Es una decisión de transición, no evidencia de que esos correos se hayan verificado antes.

```
@Column(nullable = false, columnDefinition = "boolean default true")
private boolean emailConfirmado = true;

public boolean isEmailConfirmado() { return emailConfirmado; }
public void setEmailConfirmado(boolean valor) {
    this.emailConfirmado = valor;
}
```

En una base existente, respaldá primero y agregá explícitamente la columna:

```
ALTER TABLE usuarios
ADD COLUMN IF NOT EXISTS email_confirmado boolean NOT NULL DEFAULT true;
```

Las cuentas nuevas creadas por registro público o por ADMIN deben ejecutar setEmailConfirmado(false) antes de guardarse. El boolean inicial true conserva el comportamiento de las cuentas iniciales creadas por AsaditoRgApplication; tratá esas cuentas como aprovisionadas por vos. Si la materia exige confirmar también todas las cuentas anteriores, elegí false para ellas y generá también su fila inicial en confirmaciones_email antes de habilitar el reenvío y bloquear su ingreso. Las cuentas antiguas no tienen esa fila automáticamente.

No cambies password_hash ni borres usuarios para agregar esta función. El campo emailConfirmado no es un rol: ADMIN, OPERADOR y USER siguen indicando permisos diferentes.

Paso B: entidad ConfirmacionEmail.java

```
@Entity
@Table(name = "confirmaciones_email")
public class ConfirmacionEmail {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "usuario_id", nullable = false, unique = true)
    private Usuario usuario;

    @Column(nullable = false, unique = true, length = 64)
    private String tokenHash;
    @Column(nullable = false)
    private Instant venceEn;
    private Instant usadoEn;
    @Column(nullable = false)
    private Instant ultimoEnvio;

    public ConfirmacionEmail() {}
    // Generá getters y setters.
}
```

Una fila por usuario alcanza: el reenvío reemplaza hash y vencimiento, por lo que el enlace anterior deja de servir. usadoEn es null mientras está pendiente. ultimoEnvio representa el último intento autorizado de envío; un fallo SMTP también cuenta para limitar reintentos.

Paso C: generar un token y su hash

Estos métodos privados pueden quedar en ConfirmacionEmailService. Usá java.security.SecureRandom, java.security.MessageDigest, java.security.NoSuchAlgorithmException, java.util.Base64 y java.nio.charset.StandardCharsets.

```
private String nuevoToken() {
    byte[] bytes = new byte[32];
    new SecureRandom().nextBytes(bytes);
    return Base64.getUrlEncoder().withoutPadding()
        .encodeToString(bytes);
}

private String hash(String token) {
    try {
        byte[] digest = MessageDigest.getInstance("SHA-256")
            .digest(token.getBytes(StandardCharsets.UTF_8));
        StringBuilder texto = new StringBuilder();
        for (byte b : digest) {
            texto.append(String.format("%02x", b & 0xff));
        }
        return texto.toString();
    } catch (NoSuchAlgorithmException e) {
        throw new IllegalStateException(e);
    }
}
```

Los 32 bytes dan un token de alta entropía. El texto generado tiene 43 caracteres con este formato Base64 URL sin padding; el hash SHA-256 tiene 64 caracteres hexadecimales. BCrypt sigue usándose para contraseñas. SHA-256 acá corresponde a un token aleatorio, no a una contraseña elegida por la persona.

No guardes el token original en movimientos, respuestas JSON, consola ni base de datos. Solo viaja en el enlace enviado por correo y en el POST de confirmación. Para una prueba local, obtenelo del buzón SMTP de pruebas, no agregando un endpoint que exponga tokens.

Paso D: repositorio con bloqueo de la fila

Creá ConfirmacionEmailRepository extends JpaRepository<ConfirmacionEmail, Long>. Estos métodos se invocan desde un servicio @Transactional. @Lock viene de org.springframework.data.jpa.repository; LockModeType, de javax.persistence; @Param, de org.springframework.data.repository.query.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select c from ConfirmacionEmail c join fetch c.usuario " +
       "where c.tokenHash = :hash")
Optional<ConfirmacionEmail> buscarPorHashBloqueado(
    @Param("hash") String hash);

@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select c from ConfirmacionEmail c join fetch c.usuario " +
       "where c.usuario.email = :email")
Optional<ConfirmacionEmail> buscarPorEmailBloqueado(
    @Param("email") String email);
```

El bloqueo evita que dos confirmaciones consuman el mismo registro simultáneamente. Tanto confirmar como reenviar deben usar estos métodos y volver a comprobar el estado dentro de la transacción. El registro inicial de confirmación se crea junto al usuario pendiente; el reenvío no crea otra fila para ese usuario.

Paso E: registrar la cuenta y preparar el correo

En lugar de hacer todo dentro del método privado crear de UsuarioController, extraé esa operación a un método público de ConfirmacionEmailService con @Transactional. Este es uno de los pocos servicios nuevos justificados: usuario y token se guardan juntos y el correo debe enviarse después del commit.

Contrato sugerido del método: crearCuentaPendiente(email, password, rol) retorna el token original SOLO al controlador Java que lo llamó. No lo devuelve directamente a la petición HTTP.

1. Normalizá y validá email/password con las reglas que ya tiene crear. Rechazá email repetido con 409.

2. Creá Usuario, codificá password con PasswordEncoder y ejecutá setEmailConfirmado(false).

3. Guardá el usuario. rol lo fija el código que llama al servicio: USER para registro público y OPERADOR para el endpoint de ADMIN. No lo recibas del formulario.

4. Generá un token, creá ConfirmacionEmail y guardá su hash, venceEn = Instant.now().plusSeconds(1800), usadoEn = null y ultimoEnvio = Instant.now().

5. Si integrás auditoría de creación de cuenta, distinguí al ADMIN actor del nuevo operador. En registro público podés usar actor null y un resumen de sistema sin credenciales.

6. Retorná el token al controlador; al regresar del servicio se confirma la transacción.

7. El controlador llama a CorreoService.enviarConfirmacion(emailNormalizado, token), fuera de la transacción, y devuelve un mensaje sin token ni contraseña.

No anotes el controlador exterior con @Transactional en este flujo: el envío volvería a quedar antes del commit. La anotación está en el servicio público que se invoca a través del bean inyectado. Llamar un método @Transactional desde otro método de la misma instancia no abre automáticamente otra

transacción.

Si falla SMTP, la cuenta sigue pendiente y el endpoint de reenvío permite recuperarla. Devolvé un mensaje controlado indicando que no se pudo completar el envío y que puede reenviarse; no anuncies “correo enviado” si JavaMailSender falló. No borres la cuenta ni marques confirmado para esconder el fallo. SMTP y PostgreSQL no comparten una transacción; una cola de envíos con reintentos sería una mejora posterior.

Paso F: dependencia y configuración SMTP

Agregá a build.gradle y recargá Gradle en IntelliJ:

```
implementation 'org.springframework.boot:spring-boot-starter-mail'
```

Ejemplo de propiedades para un servidor SMTP autenticado con STARTTLS; adaptá puerto y TLS a tu proveedor:

```
spring.mail.host=${SPRING_MAIL_HOST}
spring.mail.port=${SPRING_MAIL_PORT:587}
spring.mail.username=${SPRING_MAIL_USERNAME}
spring.mail.password=${SPRING_MAIL_PASSWORD}
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
spring.mail.properties.mail.smtp.starttls.required=true
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=5000
spring.mail.properties.mail.smtp.writetimeout=5000
app.mail.from=${APP_MAIL_FROM}
app.public-url=${APP_PUBLIC_URL}
```

Cargá los valores reales en Environment variables de IntelliJ; no en Git ni en esta guía. APP_PUBLIC_URL es tu origen público configurado por vos, por ejemplo http://localhost:8080 durante desarrollo y https://tu-dominio en despliegue. No construyas el enlace con un Host enviado por el cliente.

Para probar sin mandar correos reales podés usar un buzón SMTP local como Mailpit. En ese perfil de desarrollo configurá su host/puerto y desactivá auth/STARTTLS solo para ese servidor local. No copies esa configuración a un SMTP público. Abrí el buzón local, leé el enlace y probá la confirmación.

Paso G: servicio de correo sencillo

JavaMailSender y SimpleMailMessage están en org.springframework.mail y org.springframework.mail.javamail; @Value, en org.springframework.beans.factory.annotation.

```
@Service
public class CorreoService {
    @Autowired private JavaMailSender sender;
    @Value("${app.mail.from}") private String remitente;
    @Value("${app.public-url}") private String baseUrl;

    public void enviarConfirmacion(String email, String token) {
        String enlace = baseUrl + "/confirmar-email.html#token=" + token;
        SimpleMailMessage mensaje = new SimpleMailMessage();
        mensaje.setFrom(remitente);
        mensaje.setTo(email);
        mensaje.setSubject("Confirmá tu email en Asadito RG");
        mensaje.setText("Abrí este enlace y confirmá tu cuenta: "
            + enlace + "\nEl enlace vence en 30 minutos.");
        sender.send(mensaje);
    }
```

```
}
```

Configurá baseUrl sin barra final para este ejemplo. El fragmento #token no se envía en la petición GET al servidor. La página lo lee en el navegador y lo envía en el POST; evitás que aparezca como query string del GET en logs habituales.

Paso H: consumir el enlace una sola vez

El controlador ofrece POST /api/auth/confirmar-email con @RequestParam String token y delega a ConfirmacionEmailService.confirmar(token). No modifica el estado por un simple GET del enlace; algunos sistemas de correo abren enlaces automáticamente.

Dentro del servicio, con @Transactional:

```
if (token == null || !token.matches("[A-Za-z0-9_-]{43}")) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Enlace inválido o vencido");
}
ConfirmacionEmail c = confirmaciones
    .buscarPorHashBloqueado(hash(token))
    .orElseThrow(() -> new ResponseStatusException(
        HttpStatus.BAD_REQUEST, "Enlace inválido o vencido"));
Instant ahora = Instant.now();
if (c.getUsadoEn() != null || !c.getVenceEn().isAfter(ahora)) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Enlace inválido o vencido");
}
c.getUsuario().setEmailConfirmado(true);
c.setUsadoEn(ahora);
usuarios.save(c.getUsuario());
confirmaciones.save(c);
// Registrá EMAIL_CONFIRMADO sin incluir token ni hash.
```

Retorná 204 desde el controlador. Registrar el consumo y habilitar al usuario ocurre en la misma transacción. Si llegan dos POST simultáneos, solo el primero tiene éxito; el siguiente encuentra usadoEn y devuelve 400. El correo confirmado no genera una sesión nueva: la persona luego inicia sesión con su contraseña habitual.

Paso I: reenvío y pantalla de confirmación

POST /api/auth/reenviar-confirmacion recibe email. Normalizalo, usá buscarPorEmailBloqueado y, dentro de @Transactional, verificá que siga pendiente y no consumido. Si transcurrió al menos un minuto desde ultimoEnvio, reemplazá tokenHash, venceEn y ultimoEnvio; el nuevo token se envía fuera de esa transacción. El enlace viejo queda invalidado.

Respondé 202 con un mensaje general para email desconocido, cuenta ya confirmada o reenvío solicitado. Si el usuario está en el período de espera, no envíes otro correo y conservá esa respuesta general. Un limitador adicional por IP evita que se prueben miles de emails; aplicalo antes de consultar la cuenta y sin usar una IP reenviada de un proxy no confiable. El límite por cuenta se decide bajo bloqueo para evitar reenvíos simultáneos.

La pantalla confirmar-email.html puede tener un botón “Confirmar email”. Al abrirse obtiene el token del fragmento y limpia la URL con history.replaceState. Al tocar el botón:

1. GET /api/auth/csrf para obtener token CSRF y nombre de cabecera.

2. POST /api/auth/confirmar-email como URLSearchParams, con el token de email en el cuerpo y la cabecera CSRF.

3. Con 204, muestra “Email confirmado. Ya podés ingresar”. Con 400, ofrece solicitar otro enlace.

No importes session.js sin más en una página vacía: ese módulo espera los elementos de autenticación del HTML actual. Podés reutilizar su estructura de página o escribir estos dos fetch pequeños para la pantalla independiente. No guardes el token del correo en localStorage ni lo envíes a scripts de análisis.

En WebAuthorization agregá /confirmar-email.html a las páginas permitidas antes de anyRequest. /api/auth/** ya es público, pero mantiene CSRF para los POST. No desactives CSRF globalmente para resolver la confirmación.

Paso J: impedir login de cuentas pendientes

Cuando ya funcionan registro, envío, confirmación y reenvío, cambiá el retorno de WebAuthentication para usar el constructor de User con enabled:

```
return new User(usuario.getEmail(), usuario.getPassword(),
    usuario.isEmailConfirmado(), true, true, true,
    AuthorityUtils.createAuthorityList(usuario.getRol()));
```

El orden es username, password, enabled, accountNonExpired, credentialsNonExpired, accountNonLocked y authorities. Spring rechaza el login si enabled es false. Conservá el PasswordEncoder y la compatibilidad BCrypt anteriores.

Actualizá el mensaje de registro del frontend: debe pedir revisar el correo, no invitar a ingresar inmediatamente. Podés agregar emailConfirmado a /api/auth/me si la interfaz lo necesita, sin tocar los DTO de catálogo. Habilitar el correo es distinto de recuperar una contraseña; este enlace no cambia la clave ni el rol.

Probalo: una cuenta nueva no ingresa antes de confirmar; el enlace válido habilita el ingreso; el mismo enlace no sirve dos veces; un token vencido o inventado no habilita; al reenviar deja de servir el anterior; dos consumos simultáneos solo habilitan una vez; un fallo de SMTP permite reintentar sin perder la cuenta.

6. WhatsApp opcional: empezar con un bot de menú

El botón wa.me que ya tiene la aplicación abre un chat con texto preparado. Un chatbot requiere otra integración: recibir mensajes mediante un webhook y responder con la API de WhatsApp Business de Meta o un proveedor. No se consigue solo cambiando el enlace de compartir.

Para aprender, empezá con un bot por reglas, sin inteligencia artificial. Así podés explicar claramente qué entrada produce cada respuesta.

Primera versión propuesta

```
Usuario: Hola
Bot: Asadito RG
     1 - Ver cortes
     2 - Consultar precios de un corte
     3 - Abrir la calculadora
     0 - Volver al menú
Usuario: 2
Bot: Elegí un corte de la lista.
Usuario: Vacío
Bot: Comercios disponibles, de menor a mayor precio por kilo.
     Dirección y fecha de cada precio. Confirmá disponibilidad.
```

Las consultas públicas pueden reutilizar repositorios del catálogo y la selección por fecha/ID. Para responder “abrir la calculadora”, enviá un enlace a la página pública. Un estado simple MENU → ELEGIR_CORTE → MOSTRAR_PRECIOS alcanza al principio. El mensaje 0 reinicia el estado y una entrada desconocida vuelve a mostrar ayuda.

Archivos y configuración adicionales, solo si avanzás

Pieza	Responsabilidad
WhatsappWebhookController	Validar el callback, autenticar el webhook y entregar el evento al procesador.
WhatsappService	Interpretar opciones, consultar precios y enviar respuestas por la API del proveedor.
ConversacionWhatsapp, opcional	Guardar número/identificador de WhatsApp, estado y vencimiento de la conversación.
MensajeWhatsapp	Guardar el ID externo único para evitar procesar dos veces el mismo mensaje.

Para un menú público sin estado podés pedir siempre “precios vacío”; así ConversacionWhatsapp no es necesaria. Si elegís una conversación con estado, persistila con caducidad; una variable global en memoria pierde todo al reiniciar y no sirve para varias instancias.

Vas a necesitar una cuenta/aplicación Business, un número habilitado, un endpoint HTTPS público y credenciales del proveedor. En Meta se distinguen el access token para enviar, el app secret para validar la firma y el verify token que elegís para comprobar el callback. Guardalos como variables de entorno. No son contraseñas de usuarios ni se envían al frontend.

Las versiones de Graph API, límites, costos, consentimiento y reglas sobre plantillas/ventanas de atención deben comprobarse en la documentación vigente del proveedor al implementarlo. No fijes una versión de API tomada de un tutorial antiguo.

Seguridad del webhook

Si usás Meta, la verificación inicial GET recibe hub.mode, hub.verify_token y hub.challenge. Comprobá modo y verify token configurado y devolvé el challenge solo si coinciden. Este GET configura el canal; no registra usuarios ni vincula cuentas de Asadito RG.

En los POST, validá X-Hub-Signature-256 con HMAC-SHA256 y el app secret sobre los bytes EXACTOS del cuerpo recibido, antes de confiar en el JSON. Compará firmas en tiempo constante. El access token de envío no reemplaza la firma del webhook.

Las rutas externas no tienen la cookie/CSRF del navegador. Solamente si implementás este módulo, agregá antes de anyRequest:

```
.antMatchers(HttpMethod.GET,
    "/api/integraciones/whatsapp/webhook").permitAll()
.antMatchers(HttpMethod.POST,
    "/api/integraciones/whatsapp/webhook").permitAll()
```

Y excluí de CSRF únicamente ese endpoint:

```
http.csrf().ignoringAntMatchers(
    "/api/integraciones/whatsapp/webhook");
```

Conservá CSRF en registro, login, confirmación y paneles. Que el webhook sea permitAll significa que no exige JSESSIONID; la firma del proveedor pasa a ser su autenticación y debe comprobarse en el código. No habilites todo /api/** para hacer funcionar el bot.

Procesá solo los mensajes que correspondan; los callbacks también pueden traer estados de entrega. El proveedor puede repetir un evento: imponé UNIQUE sobre mensajeExternoId. Guardá el cambio de negocio y su registro de procesamiento en una transacción; no lo marques completado antes de que esa operación termine. Para enviar respuestas de forma fiable, una bandeja de salida con reintentos evita perder respuestas si la llamada externa falla; es una ampliación posterior, no necesaria para el menú de práctica.

Privacidad y presupuestos del usuario

El número que escribe por WhatsApp no es una sesión de Spring Security. No aceptes “mi usuarioId es 7” ni un email escrito en el chat como prueba de identidad. Para acceder a presupuestos privados se necesita una vinculación verificada de cuenta, con un código de un solo uso y vencimiento iniciado desde la web autenticada.

Para una primera entrega mantené el bot limitado al catálogo público y enlaces a la web. La persona inicia sesión en el navegador y allí consulta sus presupuestos. No expongas GET privados ni envíes un presupuesto solo porque quien escribe conoce su ID.

¿Puede enviar el PDF automáticamente?

Un Blob creado por reporte.js existe en el navegador. El servidor del bot no puede adjuntar ese Blob por su cuenta. Las alternativas son:

• Enviar un enlace a la web, donde la persona inicia sesión y descarga/adjunta el PDF. Es la opción más sencilla con el proyecto actual.

• Agregar generación del PDF en el servidor y cargar el documento por la API del proveedor, con autorización, almacenamiento temporal y vencimiento. Requiere Java adicional y un nuevo trabajo de backend.

Primero hacé funcionar el menú y las consultas públicas. Después evaluá vinculación de cuentas y documentos. Un chatbot no exige un modelo de lenguaje; incorporarlo agregaría un proveedor, costos y controles que no hacen falta para este menú.

7. Endpoints y permisos para revisar antes de programar

Las rutas siguientes son propuestas de esta guía; todavía no existen en el backend actual salvo registro, sesión y catálogo ya descriptos. El nombre de cada controller puede variar, pero conservá una única ruta para cada función.

Método y ruta	Quién puede usarlo	Datos principales
GET /api/admin/movimientos	ADMIN	page y size validados
POST /api/presupuestos	Sesión; siempre para el usuario actual	localId, productoId repetido y cantidad repetida
GET /api/presupuestos	Sesión; lista propia	page y size validados
GET /api/presupuestos/{id}	Solo el dueño	ID con control de propiedad
POST /api/presupuestos/{id}/verificacion	Solo el dueño	totalReal y observacion
GET /api/presupuestos/{id}/verificacion	Solo el dueño	Resultado guardado
POST /api/auth/confirmar-email	Público con CSRF	token de un solo uso
POST /api/auth/reenviar-confirmacion	Público con CSRF	email; respuesta general
GET /confirmar-email.html	Público	Pantalla que recibe el fragmento del enlace
GET/POST /api/integraciones/whatsapp/webhook	Proveedor, solo módulo opcional	Verify token o firma del proveedor

Una cuenta anónima que intenta guardar o leer presupuestos debe recibir 401 si envía una petición válida con CSRF; una escritura sin CSRF puede recibir 403 antes de comprobar sesión. Una cuenta autenticada sin permiso de auditoría recibe 403. Un presupuesto ajeno se responde como 404.

No crees endpoints públicos para insertar movimientos, elegir el rol, cambiar emailConfirmado o indicar a qué usuario pertenece un presupuesto. En el nuevo código, un 400 señala datos inválidos, 409 un conflicto de duplicado y 429 puede reservarse para límites por IP independientes del email.

El ApiErrors actual tiene un texto 409 que menciona el diagnóstico de precios. Al sumar estas tablas, reemplazá ese mensaje general por uno adecuado para todas las escrituras, por ejemplo “No se pudo guardar por un conflicto con datos existentes”. Mantené mensajes específicos para duplicados detectados antes del guardado; la restricción de la base cubre las carreras concurrentes. No expongas SQL ni tokens en el error HTTP.

8. Crear tablas y avanzar sin perder datos

Para una base de desarrollo nueva, podés crear las entidades de una etapa y dejar que ddl-auto=update cree las tablas; luego revisá la estructura en pgAdmin. Para una base con datos, usá respaldo y una migración revisada. No ejecutes scripts genéricos que borren tablas para empezar de nuevo.

El modelo Java es la referencia principal. Estos son los nombres SQL esperados con la estrategia de nombres actual de Spring Boot; comprobá las columnas realmente generadas antes de escribir consultas manuales.

Tabla	Columnas que no deberían faltar
movimientos	id bigint, usuario_id bigint nullable, fecha, accion varchar(40), entidad varchar(40), registro_id bigint, resumen varchar(300).
presupuestos	id bigint, usuario_id bigint, local_id integer, fecha, local_nombre varchar(120), local_direccion varchar(200), moneda varchar(3), total numeric(14,2).
presupuesto_detalles	id bigint, presupuesto_id bigint, producto_id integer, precio_id integer, corte_nombre varchar(120), cantidad numeric(5,1), precio_kg numeric(14,2), fecha_precio date, subtotal numeric(14,2).
verificaciones	id bigint, presupuesto_id bigint UNIQUE, fecha, total_real numeric(14,2), diferencia numeric(14,2), porcentaje numeric(18,2), resultado varchar(20), observacion varchar(300).
confirmaciones_email	id bigint, usuario_id bigint UNIQUE, token_hash varchar(64) UNIQUE, vence_en, usado_en nullable, ultimo_envio.
usuarios, columna nueva	email_confirmado boolean NOT NULL; revisar el valor de adopción para cuentas previas.

Las relaciones declaradas con @JoinColumn deben producir claves foráneas. Confirmá que exista UNIQUE en verificaciones.presupuesto_id, confirmaciones_email.usuario_id y confirmaciones_email.token_hash. No cambies los generadores ni los IDs de las tablas originales para estas ampliaciones.

Podés añadir índices para las consultas más frecuentes, una vez creadas las tablas:

```
CREATE INDEX IF NOT EXISTS idx_movimientos_fecha
ON movimientos (fecha DESC, id DESC);
CREATE INDEX IF NOT EXISTS idx_presupuestos_usuario_fecha
ON presupuestos (usuario_id, fecha DESC, id DESC);
CREATE INDEX IF NOT EXISTS idx_detalles_presupuesto
ON presupuesto_detalles (presupuesto_id);
```

Las claves UNIQUE ya crean índices de soporte en PostgreSQL; no hace falta duplicarlos para las confirmaciones y verificaciones. Para importes nuevos, una migración puede agregar CHECK(total > 0), CHECK(total_real > 0), CHECK(cantidad > 0 AND cantidad <= 1000) y un control de pasos de medio kilo. Las validaciones Java siguen siendo necesarias para devolver mensajes comprensibles.

Inventario para crear y modificar a mano

Etapa	Archivos nuevos	Archivos actuales a tocar
Movimientos	Movimiento, MovimientoRepository, MovimientoService y MovimientoController.	Controladores de cortes, locales y precios; WebAuthorization.
Presupuestos	PresupuestoGuardado, DetallePresupuesto, PresupuestoRepository y PresupuestoController.	PreciosRepository; después app.js o un módulo de Mis presupuestos.
Verificaciones	Verificacion, VerificacionRepository y VerificacionController.	Interfaz del presupuesto guardado; texto general de ApiErrors.
Email	ConfirmacionEmail, ConfirmacionEmailRepository, ConfirmacionEmailService, CorreoService y ConfirmacionEmailController.	Usuario, UsuarioController, WebAuthentication, WebAuthorization, build.gradle, application.properties y mensajes de registro.
WhatsApp opcional	WebhookController, WhatsappService y persistencia de conversación/mensajes si se elige.	WebAuthorization y variables del proveedor.

No se requiere cambiar los DTO originales, volver a introducir PDFBox ni reescribir todo el backend. Si el controlador de presupuesto te queda demasiado largo, podés extraer su guardado a un PresupuestoGuardadoService; hacelo después de comprender el flujo, conservando una única transacción exterior.

9. Pruebas concretas y criterio de terminado

Esta sección es una lista para ejecutar cuando programes los módulos. No son funcionalidades ni pruebas que ya estén implementadas en el proyecto por haber leído esta guía.

Movimientos

• Un OPERADOR publica un precio válido: quedan precio y movimiento con su usuario.

• Un precio negativo o fecha futura falla sin crear movimiento.

• Si forzás una falla de auditoría en una prueba, tampoco queda el cambio principal.

• ADMIN lista movimientos paginados; USER y OPERADOR reciben 403.

• No hay contraseñas, hashes ni tokens dentro del resumen.

Presupuestos

• 1,5 kg a $10.000,01 produce $15.000,02; no aparece un error de coma flotante en la suma nueva.

• 0,25 kg, valores negativos, más de 100 filas, productos repetidos o listas de longitudes distintas reciben 400.

• Un corte/local inactivo o sin precio vigente impide guardar el presupuesto completo.

• Modificar un precio o un nombre del catálogo después del guardado no altera el historial.

• Un total o usuarioId agregado por el cliente no cambia el cálculo ni la propiedad.

• Usuario B no puede listar ni consultar el presupuesto de A, incluso adivinando el ID.

• El detalle y la cabecera se guardan juntos, y un error revierte ambos y su movimiento.

Verificaciones

• Se obtienen ACORDE, SOBREESTIMADO y SUBESTIMADO con los ejemplos del capítulo 4.

• El límite exacto del 5 % es ACORDE; algo mayor queda fuera aunque el porcentaje mostrado se redondee.

• No se divide por cero y se rechazan importes fuera de rango o con decimales excedentes.

• Una segunda verificación devuelve 409; dos solicitudes simultáneas dejan una sola fila.

• B no verifica ni consulta el resultado del presupuesto de A.

Emails

• El registro nuevo genera usuario pendiente y hash de token, nunca el token original en la base.

• No se inicia sesión antes de confirmar. Después de confirmar se usa el login habitual y el mismo hash de contraseña.

• Vencido, inventado, ya usado o reemplazado por reenvío no habilita la cuenta.

• Dos confirmaciones simultáneas no consumen el mismo token dos veces.

• Un GET del enlace no confirma la cuenta; se requiere el POST explícito con CSRF.

• El reenvío tiene espera mínima y respuesta general para no revelar cuentas.

• Si SMTP falla, no se marca confirmado y existe un camino para reenviar.

• Las cuentas previas que decidiste conservar habilitadas siguen ingresando; la migración no modifica roles ni contraseñas.

WhatsApp, solo si lo implementás

• Una firma inválida no procesa mensajes; un challenge con verify token incorrecto se rechaza.

• Un evento repetido no repite cambios; un estado de entrega no se interpreta como mensaje del usuario.

• El bot entrega precios públicos por fecha/ID y vuelve al menú con 0.

• Un número sin vincular no recibe presupuestos privados.

• CSRF sigue funcionando en todas las rutas del navegador.

Usá primero una base de prueba y un buzón SMTP local. Cuando una etapa esté lista, ejecutá bash gradlew test y bash gradlew build, y repetí una prueba manual con usuarios de roles distintos. Agregá tests de propiedad, tokens y transacciones antes de continuar; aportan más que comprobar solo el estado 200.

10. Cómo explicarlo y lecturas para continuar

Podés defender el diseño con estas ideas: movimientos responde quién hizo qué; presupuestos guarda una fotografía de una estimación; detalles representa cada corte y su cantidad; verificaciones compara esa fotografía con un pago declarado; confirmaciones_email prueba el control de un buzón para una cuenta nueva; WhatsApp es un canal adicional que no reemplaza la autenticación.

Antes de mostrar una clase, ubicá su responsabilidad y seguí una petición completa. Ejemplo: POST presupuesto → usuario de la sesión → validaciones → precios de la base → cabecera y detalles → movimiento → commit → respuesta. Explicá por qué una falla no debe dejar un presupuesto a medio guardar.

Para email: registro pendiente → token aleatorio → hash en la base → envío tras commit → POST de confirmación → consumo y habilitación → login normal. El enlace no contiene la contraseña y no cambia el rol.

Lecturas oficiales recomendadas para consultar al codificar; la integración de WhatsApp debe ajustarse a las versiones y políticas vigentes en ese momento:

• Spring Boot 2.7.14, correo: https://docs.spring.io/spring-boot/docs/2.7.14/reference/html/io.html#io.email

• Spring Data JPA 2.7, métodos de repositorio y bloqueo: https://docs.spring.io/spring-data/jpa/docs/2.7.14/reference/html/

• Spring Security 5.7: https://docs.spring.io/spring-security/reference/5.7/

• BigDecimal, Java 11: https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/math/BigDecimal.html

• PostgreSQL, restricciones: https://www.postgresql.org/docs/current/ddl-constraints.html

• WhatsApp Cloud API: https://developers.facebook.com/docs/whatsapp/cloud-api/

• Meta, webhooks: https://developers.facebook.com/docs/graph-api/webhooks/

El material ModificaClient.pdf sigue siendo la referencia docente para tus sesiones. Esta propuesta agrega módulos alrededor de esa base y conserva su enfoque. Anotá tus decisiones finales —tolerancia, migración de cuentas anteriores y alcance del bot— antes de programarlas para poder justificarlas después.